

MamaCare — Implementation Plan & Developer Guide

Maternal Health Appointment Tracker

PART A: IMPLEMENTATION PLAN

1. Project Setup & Foundation (Day 1)

1.1 Repository Initialization

```
# Create new Next.js project with TypeScript
npx create-next-app@latest mamacare --typescript --tailwind --eslint --app --src-dir --import-alias "@/
↵  * "
cd mamacare

# Install core dependencies
npm install @supabase/supabase-js @supabase/auth-helpers-nextjs
npm install idb date-fns
npm install zod react-hook-form @hookform/resolvers
npm install lucide-react
npm install next-pwa
npm install -D @types/node @types/react @types/react-dom
npm install -D jest @testing-library/react @testing-library/jest-dom @testing-library/user-event
npm install -D playwright

# Initialize Git
git init
git add .
git commit -m "Initial commit: MamaCare project setup"
```

1.2 Environment Configuration

```
# Create environment files
touch .env.local .env.example
```

.env.local:

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key

# App
NEXT_PUBLIC_APP_NAME=MamaCare
NEXT_PUBLIC_APP_URL=http://localhost:3000

# SMS/WhatsApp (Twilio)
TWILIO_ACCOUNT_SID=your-account-sid
TWILIO_AUTH_TOKEN=your-auth-token
TWILIO_PHONE_NUMBER=your-twilio-number
```

```

TWILIO_WHATSAPP_NUMBER=your-whatsapp-number

```

1.3 Supabase Project Setup

- 1. Go to supabase.com and create new project
- 2. Copy project URL and anon key to `.env.local`
- 3. Run SQL migrations (see Section 8.3 of PRD)
- 4. Enable Phone Auth in Authentication → Providers → Phone
- 5. Configure RLS policies

1.4 Next.js Configuration

```

// next.config.js
const withPWA = require('next-pwa')({
  dest: 'public',
  register: true,
  skipWaiting: true,
  disable: process.env.NODE_ENV === 'development',
  runtimeCaching: [
    {
      urlPattern: /^https:\/\/.*\.supabase\.co\/.*$/,
      handler: 'NetworkFirst',
      options: {
        cacheName: 'supabase-cache',
        expiration: { maxEntries: 50, maxAgeSeconds: 86400 }
      }
    }
  ]
});

/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
  images: {
    domains: ['your-project.supabase.co']
  }
};

module.exports = withPWA(nextConfig);

```

2. Week 1 Implementation Schedule

Day 1: Project Foundation

Task	Time	Files
Initialize Next.js project	30 min	package.json, tsconfig.json
Configure Tailwind + PWA	30 min	tailwind.config.ts, next.config.js
Set up Supabase client	30 min	lib/db/supabase.ts

Table 1 – continued

Task	Time	Files
Set up IndexedDB	45 min	lib/db/indexeddb.ts
Create type definitions	30 min	types/index.ts
Create folder structure	15 min	All directories
Total	~3 hours	

Key Files to Create:

```

lib/
  db/
    supabase.ts      # Supabase client singleton
    indexeddb.ts     # IndexedDB setup with idb
  utils/
    date.ts          # date-fns wrappers
    encryption.ts     # SubtleCrypto helpers
  constants/
    decision-tree.ts  # Decision tree rules
    visit-schedule.ts # Recommended visit intervals
  types/
    index.ts          # All TypeScript interfaces

```

Day 2: Authentication System

Task	Time	Files
Phone OTP login form	1.5 hr	app/(auth)/login/page.tsx
OTP verification	1 hr	app/(auth)/login/verify/ page.tsx
Auth context/provider	1 hr	app/providers/ AuthProvider.tsx
Role-based route guards	1 hr	app/middleware.ts
Session persistence	30 min	hooks/useAuth.ts
Total	~5 hours	

Auth Flow:

```

User enters phone → Supabase sends OTP → User enters code →
JWT stored in memory → Role checked → Redirect to dashboard

```

Day 3: Patient Registration

Task	Time	Files
Patient form component	2 hr	components/forms/ PatientForm.tsx
Form validation (Zod)	1 hr	lib/validation/patient.ts

Table 3 – continued

Task	Time	Files
Photo capture/upload	1 hr	components/forms/ PhotoCapture.tsx
IndexedDB patient store	1 hr	hooks/useOfflineStorage.ts
Patient list page	1 hr	app/(dashboard)/chw/ patients/page.tsx
Total	~6 hours	

Day 4: Visit Logging

Task	Time	Files
Visit form with vitals	2 hr	components/forms/VisitForm.tsx
Visit history timeline	1.5 hr	components/VisitTimeline.tsx
Patient detail page	1.5 hr	app/(dashboard)/chw/patients/[id]/ page.tsx
Offline visit storage	1 hr	Update useOfflineStorage.ts
Total	~6 hours	

Day 5: Appointment Scheduling

Task	Time	Files
Appointment form	1.5 hr	components/forms/ AppointmentForm.tsx
Calendar view	2 hr	components/calendar/ AppointmentCalendar.tsx
Appointment list	1 hr	app/(dashboard)/chw/appointments/ page.tsx
Auto-calculate visit intervals	1 hr	lib/utils/visit-schedule.ts
Total	~5.5 hours	

Day 6: Decision Tree

Task	Time	Files
Symptom checklist UI	1.5 hr	components/decision-tree/ SymptomChecker.tsx
Rule engine	2 hr	lib/decision-tree/engine.ts
Triage result display	1 hr	components/decision-tree/ TriageResult.tsx
Integration with visit form	1 hr	Update VisitForm.tsx
Total	~5.5 hours	

Day 7: Integration & Testing

Task	Time	Files
Wire all pages together	2 hr	Multiple
Offline indicator	1 hr	components/offline/ OfflineIndicator.tsx
CHW dashboard	2 hr	app/(dashboard)/chw/page.tsx
Basic testing	2 hr	Test files
Total	~7 hours	

3. Week 2 Implementation Schedule

Day 8: At-Risk Detection

Task	Time	Files
Risk assessment logic	2 hr	lib/risk/assessment.ts
At-risk patient list	1.5 hr	app/(dashboard)/chw/at-risk/ page.tsx
Risk badge components	1 hr	components/RiskBadge.tsx
Alert notifications	1.5 hr	components/AlertBanner.tsx
Total	~6 hours	

Day 9: Nurse Dashboard

Task	Time	Files
Dashboard layout	1 hr	app/(dashboard)/nurse/layout.tsx
Stats cards	1.5 hr	components/dashboard/ StatsCards.tsx
Patient table with filters	2 hr	components/dashboard/ PatientTable.tsx
Export functionality	1.5 hr	app/api/reports/export/route.ts
Total	~6 hours	

Day 10: Sync Engine

Task	Time	Files
Sync queue management	2 hr	lib/sync/queue.ts
Push changes to Supabase	2 hr	lib/sync/push.ts
Pull changes from Supabase	2 hr	lib/sync/pull.ts
Conflict resolution	1.5 hr	lib/sync/conflict.ts
Background sync trigger	1 hr	hooks/useSync.ts
Total	~8.5 hours	

Day 11: SMS/WhatsApp Reminders

Task	Time	Files
Twilio integration	1.5 hr	<code>lib/sms/twilio.ts</code>
Reminder API route	1.5 hr	<code>app/api/reminders/route.ts</code>
Scheduled reminders	2 hr	<code>lib/reminders/scheduler.ts</code>
Reminder templates	1 hr	<code>lib/reminders/templates.ts</code>
Delivery tracking	1 hr	Update appointment schema
Total	~7 hours	

Day 12: Testing & Bug Fixes

Task	Time	Files
Unit tests for decision tree	2 hr	<code>__tests__/decision-tree.test.ts</code>
Unit tests for sync engine	2 hr	<code>__tests__/sync.test.ts</code>
E2E tests (Playwright)	2 hr	<code>e2e/</code>
Offline scenario testing	2 hr	Manual + automated
Bug fixes	2 hr	Various
Total	~10 hours	

Day 13: PWA Optimization

Task	Time	Files
Service worker tuning	1.5 hr	<code>public/sw.js</code>
Manifest optimization	30 min	<code>public/manifest.json</code>
Icon generation	30 min	<code>public/icons/</code>
Performance audit	1 hr	Lighthouse
Cache strategies	1 hr	Workbox config
Total	~4.5 hours	

Day 14: Demo Preparation

Task	Time	Files
Demo script	1 hr	<code>docs/demo-script.md</code>
Demo data seeding	1 hr	<code>scripts/seed.ts</code>
Final testing	2 hr	All flows
Documentation	1 hr	<code>README.md</code>
Total	~5 hours	

4. Total Effort Estimate

Phase	Days	Hours	Focus
Foundation	1	3	Setup, config, types
Week 1	6	30	Core CHW features
Week 2	6	41	Advanced features, sync, testing
Total	13	~74 hours	

PART B: DEVELOPER GUIDE

5. Development Environment Setup

5.1 Prerequisites

- Node.js 18+
- npm or yarn
- Git
- Supabase CLI (optional but recommended)
- Android device or emulator for PWA testing

5.2 Local Development

```
# Clone repository
git clone <repo-url>
cd mamacare

# Install dependencies
npm install

# Set up environment
cp .env.example .env.local
# Edit .env.local with your Supabase credentials

# Run development server
npm run dev

# Open http://localhost:3000
```

5.3 Supabase Local Development (Optional)

```
# Install Supabase CLI
npm install -g supabase

# Start local Supabase
supabase start

# Link to your project
supabase link --project-ref your-project-ref
```

```
# Run migrations
supabase db reset
```

6. Code Architecture Guide

6.1 Directory Structure

```
mamacare/
├─ app/                                # Next.js App Router
│   ├─ (auth)/                        # Auth route group
│   │   ├─ login/
│   │   │   └─ page.tsx              # Phone login
│   │   └─ layout.tsx               # Auth layout (no sidebar)
│   ├─ (dashboard)/                  # Dashboard route group
│   │   ├─ chw/                     # CHW pages
│   │   │   ├─ page.tsx             # CHW home
│   │   │   └─ patients/
│   │   │       ├─ page.tsx         # Patient list
│   │   │       └─ new/
│   │   │           └─ page.tsx     # New patient
│   │   │               └─ [id]/
│   │   │                   └─ page.tsx # Patient detail
│   │   │   ├─ visits/
│   │   │   ├─ appointments/
│   │   │   └─ decision-tree/
│   │   │       └─ at-risk/
│   │   └─ nurse/                   # Nurse pages
│   │       ├─ page.tsx             # Nurse dashboard
│   │       └─ patients/
│   └─ api/                         # API routes
│       ├─ auth/
│       │   └─ otp/
│       │       └─ route.ts
│       └─ sync/
│           └─ route.ts
│       └─ reminders/
│           └─ route.ts
│   └─ layout.tsx                   # Root layout
│       └─ globals.css
├─ components/                      # React components
│   ├─ ui/                         # Base UI (buttons, inputs, cards)
│   ├─ forms/                      # Form components
│   ├─ dashboard/                  # Dashboard widgets
│   ├─ decision-tree/              # Symptom checker
│   └─ offline/                   # Offline indicators
├─ hooks/                          # Custom React hooks
│   ├─ useAuth.ts
│   └─ useOfflineStorage.ts
```



```

|   └─ useSync.ts
|   └─ useDecisionTree.ts
└─ lib/                                # Utility libraries
    └─ db/
        └─ supabase.ts                # Supabase client
        └─ indexeddb.ts               # IndexedDB setup
    └─ sync/
        └─ engine.ts                  # Sync orchestrator
        └─ queue.ts                   # Sync queue
        └─ push.ts                     # Push to server
        └─ pull.ts                     # Pull from server
        └─ conflict.ts                 # Conflict resolution
    └─ decision-tree/
        └─ rules.ts                   # Rule definitions
        └─ engine.ts                   # Rule evaluation
    └─ risk/
        └─ assessment.ts               # Risk scoring
    └─ reminders/
        └─ scheduler.ts               # Reminder scheduling
        └─ templates.ts               # Message templates
        └─ twilio.ts                  # Twilio integration
    └─ sms/
        └─ twilio.ts                  # SMS sending
    └─ utils/
        └─ date.ts                     # Date utilities
        └─ encryption.ts              # Data encryption
        └─ validation.ts              # Zod schemas
    └─ constants/
        └─ decision-tree.ts           # Decision tree constants
        └─ visit-schedule.ts          # Visit schedule rules
└─ types/
    └─ index.ts                       # TypeScript interfaces
└─ public/
    └─ manifest.json                  # PWA manifest
    └─ sw.js                          # Service worker
    └─ icons/                          # App icons
└─ __tests__/                          # Unit tests
└─ e2e/                                # Playwright tests
└─ scripts/                            # Utility scripts
    └─ seed.ts                        # Demo data

```

6.2 Key Architectural Patterns

Pattern 1: Offline-First Data Flow

```

// Every data mutation follows this pattern:

async function createPatient(patientData: PatientInput) {
  // 1. Validate
  const validated = patientSchema.parse(patientData);

```

```

// 2. Generate ID locally
const patient: Patient = {
  id: crypto.randomUUID(),
  ...validated,
  created_at: new Date().toISOString(),
  updated_at: new Date().toISOString(),
  synced_at: null,
  is_deleted: false
};

// 3. Save to IndexedDB immediately
await db.put('patients', patient);

// 4. Add to sync queue
await addToSyncQueue({
  id: crypto.randomUUID(),
  table: 'patients',
  operation: 'INSERT',
  data: patient,
  status: 'pending',
  created_at: new Date().toISOString(),
  retry_count: 0
});

// 5. Update UI state
dispatch({ type: 'ADD_PATIENT', payload: patient });

// 6. Trigger sync if online
if (navigator.onLine) {
  syncEngine.sync();
}

return patient;
}

```

Pattern 2: Optimistic UI Updates

```

// Show result immediately, handle errors gracefully

function usePatientMutations() {
  const { dispatch } = useAppContext();

  const addPatient = async (data: PatientInput) => {
    // Optimistically add to UI
    const tempPatient = { ...data, id: 'temp-' + Date.now() };
    dispatch({ type: 'ADD_PATIENT', payload: tempPatient });

    try {
      const saved = await createPatient(data);
      // Replace temp with real
      dispatch({ type: 'UPDATE_PATIENT', payload: saved });
    }
  };
}

```

```

        return saved;
    } catch (error) {
        // Revert on failure
        dispatch({ type: 'DELETE_PATIENT', payload: tempPatient.id });
        throw error;
    }
};

return { addPatient };
}

```

Pattern 3: Sync Status Indicators

```

// Always show sync state to user

function SyncStatus() {
    const { isOnline, isSyncing, pendingCount, lastSyncTime } = useSync();

    if (!isOnline) {
        return (
            <Badge variant="secondary">
                <WifiOff className="w-3 h-3 mr-1" />
                Offline {pendingCount} pending
            </Badge>
        );
    }

    if (isSyncing) {
        return (
            <Badge variant="warning">
                <Loader2 className="w-3 h-3 mr-1 animate-spin" />
                Syncing...
            </Badge>
        );
    }

    return (
        <Badge variant="success">
            <Check className="w-3 h-3 mr-1" />
            Synced {formatDistanceToNow(lastSyncTime)} ago
        </Badge>
    );
}

```

7. Component Development Guide

7.1 Building a Form Component

Step 1: Define Zod Schema

```
// lib/utils/validation.ts
import { z } from 'zod';

export const patientSchema = z.object({
  full_name: z.string().min(2, "Name must be at least 2 characters"),
  age: z.number().min(10).max(60),
  phone: z.string().regex(/^+?[1-9]\d{1,14}$/, "Invalid phone number"),
  village: z.string().min(1, "Village is required"),
  address: z.string().optional(),
  emergency_contact: z.string().optional(),
  emergency_phone: z.string().optional(),
  edd: z.string().datetime(),
  gravida: z.number().min(0).default(0),
  para: z.number().min(0).default(0),
  known_risk_factors: z.array(z.string()).default([])
});

export type PatientInput = z.infer<typeof patientSchema>;
```

Step 2: Create Form Component

```
// components/forms/PatientForm.tsx
'use client';

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { patientSchema, PatientInput } from '@lib/utils/validation';
import { usePatientMutations } from '@hooks/useOfflineStorage';
import { Button } from '@components/ui/Button';
import { Input } from '@components/ui/Input';
import { Select } from '@components/ui/Select';

export function PatientForm({ onSuccess }: { onSuccess?: () => void }) {
  const { addPatient, isLoading } = usePatientMutations();

  const form = useForm<PatientInput>({
    resolver: zodResolver(patientSchema),
    defaultValues: {
      gravida: 0,
      para: 0,
      known_risk_factors: []
    }
  });

  const onSubmit = async (data: PatientInput) => {
    try {
      await addPatient(data);
      form.reset();
      onSuccess?.();
    } catch (error) {
```

```

    // Error handled by hook
  }
};

return (
  <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4">
    <div className="grid grid-cols-1 gap-4">
      <Input
        label="Full Name"
        {...form.register('full_name')}
        error={form.formState.errors.full_name?.message}
      />

      <div className="grid grid-cols-2 gap-4">
        <Input
          label="Age"
          type="number"
          {...form.register('age', { valueAsNumber: true })}
          error={form.formState.errors.age?.message}
        />

        <Input
          label="Phone Number"
          {...form.register('phone')}
          error={form.formState.errors.phone?.message}
          placeholder="+232XXXXXXX"
        />
      </div>

      <Input
        label="Village"
        {...form.register('village')}
        error={form.formState.errors.village?.message}
      />

      <Input
        label="Estimated Delivery Date"
        type="date"
        {...form.register('edd')}
        error={form.formState.errors.edd?.message}
      />

      <div className="grid grid-cols-2 gap-4">
        <Input
          label="Gravida (Total Pregnancies)"
          type="number"
          {...form.register('gravida', { valueAsNumber: true })}
        />

        <Input
          label="Para (Live Births)"

```

```

        type="number"
        {...form.register('para', { valueAsNumber: true })}
      />
    </div>
  </div>

  <Button type="submit" isLoading={isLoading} className="w-full">
    Register Patient
  </Button>
</form>
);
}

```

7.2 Building a Dashboard Widget

```

// components/dashboard/StatsCards.tsx
'use client';

import { useMemo } from 'react';
import { useAppContext } from '@app/providers/AppProvider';
import { Card } from '@components/ui/Card';
import { Users, Calendar, AlertTriangle, CheckCircle } from 'lucide-react';

export function StatsCards() {
  const { patients, appointments, atRiskPatients } = useAppContext();

  const stats = useMemo(() => {
    const today = new Date();
    const upcomingAppointments = appointments.filter(
      apt => new Date(apt.appointment_date) >= today && apt.status === 'scheduled'
    );

    const missedAppointments = appointments.filter(
      apt => new Date(apt.appointment_date) < today && apt.status === 'missed'
    );

    return {
      totalPatients: patients.length,
      upcomingAppointments: upcomingAppointments.length,
      missedAppointments: missedAppointments.length,
      atRiskCount: atRiskPatients.length
    };
  }, [patients, appointments, atRiskPatients]);

  const cards = [
    {
      title: 'Total Patients',
      value: stats.totalPatients,
      icon: Users,
      color: 'blue'
    },

```

```

    {
      title: 'Upcoming Appointments',
      value: stats.upcomingAppointments,
      icon: Calendar,
      color: 'green'
    },
    {
      title: 'Missed Appointments',
      value: stats.missedAppointments,
      icon: AlertTriangle,
      color: 'yellow'
    },
    {
      title: 'At-Risk Patients',
      value: stats.atRiskCount,
      icon: AlertTriangle,
      color: 'red'
    }
  ]

  return (
    <div className="grid grid-cols-2 gap-4">
      {cards.map(card => (
        <Card key={card.title} className="p-4">
          <div className="flex items-center justify-between">
            <div>
              <p className="text-sm text-gray-500">{card.title}</p>
              <p className="text-2xl font-bold">{card.value}</p>
            </div>
            <card.icon className={`w-8 h-8 text-${card.color}-500`} />
          </div>
        </Card>
      ))}
    </div>
  );
}

```

8. Testing Guide

8.1 Unit Tests

```

// __tests__/decision-tree.test.ts
import { evaluateDecisionTree } from '@lib/decision-tree/engine';

describe('Decision Tree Engine', () => {
  it('should flag red for vaginal bleeding', () => {
    const result = evaluateDecisionTree({
      symptoms: ['vaginal_bleeding'],
      vitalSigns: {}
    });
  });
}

```

```

    expect(result.level).toBe('red');
    expect(result.referralNeeded).toBe(true);
  });

  it('should flag yellow for headache + vision changes', () => {
    const result = evaluateDecisionTree({
      symptoms: ['headache', 'vision_changes'],
      vitalSigns: { systolic: 130 }
    });

    expect(result.level).toBe('yellow');
    expect(result.referralNeeded).toBe(true);
  });

  it('should flag red for severe hypertension', () => {
    const result = evaluateDecisionTree({
      symptoms: [],
      vitalSigns: { systolic: 170, diastolic: 115 }
    });

    expect(result.level).toBe('red');
  });

  it('should return green for routine symptoms', () => {
    const result = evaluateDecisionTree({
      symptoms: ['mild_nausea'],
      vitalSigns: { systolic: 110, diastolic: 70 }
    });

    expect(result.level).toBe('green');
    expect(result.referralNeeded).toBe(false);
  });
});

```

8.2 E2E Tests

```

// e2e/patient-registration.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Patient Registration', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
    // Login as CHW
    await page.fill('[name="phone"]', '+23276123456');
    await page.click('button[type="submit"]');
    // Enter OTP (use test OTP in dev)
    await page.fill('[name="otp"]', '123456');
    await page.click('button[type="submit"]');
  });
});

```



```

test('should register a new patient offline', async ({ page, context }) => {
  // Go offline
  await context.setOffline(true);

  // Navigate to new patient
  await page.click('text=New Patient');

  // Fill form
  await page.fill('[name="full_name"]', 'Aminata Conteh');
  await page.fill('[name="age"]', '24');
  await page.fill('[name="phone"]', '+23276111111');
  await page.fill('[name="village"]', 'Koidu');
  await page.fill('[name="edd"]', '2026-12-15');

  // Submit
  await page.click('button[type="submit"]');

  // Should show success without network
  await expect(page.locator('text=Patient registered')).toBeVisible();

  // Verify in IndexedDB
  const patients = await page.evaluate(() => {
    return new Promise((resolve) => {
      const request = indexedDB.open('MamaCareDB');
      request.onsuccess = () => {
        const db = request.result;
        const tx = db.transaction('patients', 'readonly');
        const store = tx.objectStore('patients');
        const getAll = store.getAll();
        getAll.onsuccess = () => resolve(getAll.result);
      };
    });
  });

  expect(patients).toHaveLength(1);
  expect(patients[0].full_name).toBe('Aminata Conteh');

  // Go online and verify sync
  await context.setOffline(false);
  await page.waitForTimeout(3000); // Wait for sync

  // Check sync queue is empty
  const queue = await page.evaluate(() => {
    return new Promise((resolve) => {
      const request = indexedDB.open('MamaCareDB');
      request.onsuccess = () => {
        const db = request.result;
        const tx = db.transaction('syncQueue', 'readonly');
        const store = tx.objectStore('syncQueue');
        const getAll = store.getAll();
        getAll.onsuccess = () => resolve(getAll.result);
      };
    });
  });

```

```

    };
  });
});

expect(queue).toHaveLength(0);
});
});

```

8.3 Offline Testing Checklist

Test	Steps	Expected Result
Offline registration	Turn off WiFi → Register patient	Patient saved locally, success shown
Offline visit log	Log visit while offline	Visit queued for sync
Sync on reconnect	Turn WiFi back on	Auto-sync triggers, queue clears
Conflict resolution	Edit same record on two devices	Last write wins, no data loss
App reload offline	Close app, reopen offline	All data still available
Extended offline	Stay offline for 7 days	All functions work, queue grows

9. Deployment Guide

9.1 Vercel Deployment

```

# Install Vercel CLI
npm i -g vercel

# Login
vercel login

# Deploy
vercel --prod

# Set environment variables
vercel env add NEXT_PUBLIC_SUPABASE_URL
vercel env add NEXT_PUBLIC_SUPABASE_ANON_KEY
vercel env add SUPABASE_SERVICE_ROLE_KEY

```

9.2 Supabase Production Setup

1. Create new Supabase project for production
2. Run all migrations
3. Configure RLS policies
4. Set up phone auth provider
5. Configure CORS for your Vercel domain

9.3 Environment-Specific Config

```
// lib/config.ts
export const config = {
  supabase: {
    url: process.env.NEXT_PUBLIC_SUPABASE_URL!,
    anonKey: process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  },
  app: {
    name: process.env.NEXT_PUBLIC_APP_NAME || 'MamaCare',
    url: process.env.NEXT_PUBLIC_APP_URL || 'http://localhost:3000'
  },
  sync: {
    interval: 30000, // 30 seconds
    retryAttempts: 3,
    retryDelay: 5000
  },
  reminders: {
    enabled: true,
    channels: ['sms', 'whatsapp']
  }
};
```

10. Troubleshooting Guide

Common Issues & Solutions

Issue	Cause	Solution
IndexedDB not working	Browser private mode	Use normal browsing mode
Sync failing	CORS issue	Check Supabase CORS settings
OTP not received	Twilio config	Verify Twilio credentials and phone number format
PWA not installable	Manifest issue	Check manifest.json validity
Slow performance	Large dataset	Implement pagination, virtual lists
Data conflicts	Concurrent edits	Review conflict resolution logic
Service worker errors	Cache corruption	Unregister and re-register SW

Debug Commands

```
# Check IndexedDB in browser console
const db = await openDB('MamaCareDB', 1);
const patients = await db.getAll('patients');
console.log(patients);

# Check sync queue
const queue = await db.getAll('syncQueue');
console.log(queue);
```

```
# Force sync
window.dispatchEvent(new Event('online'));

# Clear all data (development only)
indexedDB.deleteDatabase('MamaCareDB');
```

11. Performance Optimization Checklist

- Implement virtual scrolling for patient lists (>50 items)
 - Use React.memo for list items
 - Lazy load decision tree rules
 - Compress images before storage
 - Use tree-shaking (import specific functions)
 - Implement pagination for visit history
 - Cache decision tree results
 - Debounce search inputs
 - Use Web Workers for heavy computations
 - Implement request batching for sync
-

12. Security Checklist

- All API routes validate authentication
 - RLS policies restrict data access
 - Patient data encrypted in IndexedDB
 - No PHI in URL parameters
 - Auto-logout after 15 min inactivity
 - Secure session storage (not localStorage)
 - Rate limiting on auth endpoints
 - Input sanitization on all forms
 - HTTPS enforced in production
 - Regular dependency audits (npm audit)
-

Document Version: 1.0 Last Updated: June 2026